Режимы отображения вьюпорта и политика представления

FemboyEngine2D 0.5.0 содержит шесть режимов отображения вьюпорта, одну лестницу политик коллизий, один компонент масок рендера и один унифицированный источник света. Вместе они отвечают на вопросы, которые возникают у 2D-сцены во время авторинга: правильна ли картинка в релизе, правильно ли разложен арт, совпадает ли настройка коллизий с тем, что чувствует игрок, что стоит кадру, какие лампы достают до спрайта и согласовано ли запечённое поле расстояний с рисунком.

Это руководство — авторский справочник по этим системам. Здесь описаны точная цветовая легенда каждого режима, когда его применять, как читать плохой результат и что менять, а также данные авторинга, стоящие за диагностикой: лестница режимов коллизий, маски рендера, три типа источников света и поведение их конусов, жизненный цикл статических и «спящих статических» источников, остановка времени и палитра SDF-примитивов.

Режим вьюпорта — это настройка окружения редактора, а не настройка сцены. Она хранится в файле проекта и дублируется в INI редактора, никогда не влияет на релизную сборку, а сама сцена при переключении режимов остаётся нетронутой. Ключей два: `viewport_view_mode` (по умолчанию `0`, Standard) и `viewport_complexity_budget` (по умолчанию `8`). Полный список ключей, их значения по умолчанию и ограничения — в [справочнике настроек редактора](EditorSettingsReference_RU.md).

1. Где находятся элементы управления

Переключатель режима — выпадающий список на панели вьюпорта, слева от кнопок Move, Scale, World/Local и пошаговых стрелок. В закрытом состоянии он показывает полное название активного режима.

Элемент	Где	Поведение
Список режимов	Панель вьюпорта, левее кнопок гизмо	Перечисляет шесть режимов в порядке перечисления; выбор применяется сразу и пишет в строку состояния `view mode: <короткое имя>`
Карточка помощи	Появляется только при наведении на закрытый список	Название, краткое описание, два-три абзаца помощи и цветовая легенда режима
Viewport mode	Панель Editor Settings, раздел Display	Те же шесть режимов в виде подписанного комбо; всплывающая подсказка — краткое описание
Complexity budget	Панель Editor Settings, раздел Display	Ползунок от 1 до 64; опорное значение нормализации тепловой шкалы Shader Complexity

Карточка помощи появляется только при наведении. Она возникает рядом с курсором с небольшой задержкой, имеет ширину 480 пикселей и скрывается, пока открыт сам список, потому что тогда состояние наведения принадлежит строкам списка. Читайте её, когда не уверены, какой режим выбрать: текст в ней тот же, что и в этом руководстве.

Предсказуемость переключателя держится на трёх правилах:

- Порядок в списке, порядок перебора и сохранённое значение используют одно перечисление, поэтому `0` — это всегда Standard, а `5` — всегда SDF Visualizer.
- Новые режимы добавляются в конец, а не вставляются в середину, поэтому сохранённый проект сохраняет смысл между версиями движка.
- Недопустимое сохранённое значение при чтении проекта приводится к Standard, поэтому правка файла руками не может оставить вьюпорт пустым.

2. Шесть режимов

Два режима показывают саму сцену: Standard и Unlit. Четыре режима — это диагностические наложения: Collision Overlap, Shader Complexity, Light Complexity и SDF Visualizer. Наложение заменяет цвет спрайта диагностическим оттенком, поэтому картинка рассказывает о данных, а не об арте. Возврат в Standard доказывает, что исправление изменило релизную картинку, а не только наложение.

Краткая сводка:

Название в списке	**Короткое имя**	**На какой вопрос отвечает**	**Наложение**	**Что читает**
Standard (Lit)	Standard	Правильна ли релизная картинка?	Нет	Свет
Unlit / Wireframe	Unlit	Правильны ли арт, UV-прямоугольник и пивот?	Нет	Границы коллайдеров
Collision Overlap	Collision	Соответствует ли каждый коллайдер задуманному уровню участия?	Да	Коллайдеры
Shader Complexity / Overdraw	Overdraw	Какие пиксели платят за перекрытия и свет?	Да	Свет, коллайдеры
Light Complexity	Lights	Сколько ламп достаёт до каждого спрайта?	Да	Свет
SDF Visualizer	SDF	Согласовано ли запечённое поле расстояний с рисунком?	Да	Поле SDF, коллайдеры

2.1 Standard (Lit)

Эталонная картинка. Спрайты сохраняют авторский цвет, а весь свет применяется ровно так же, как в релизной сборке: заполняющий ambient, глобальные лампы, всенаправленные лампы, конусы spot и arc, затухание по расстоянию, SDF-тени от запечённого поля и от динамических окклюдеров, ambient occlusion и тонмаппинг.

Легенда и точные оттенки:

Строка легенды	RGB	Hex	Смысл на картинке
Authored sprite colour	180, 190, 210	#B4BED2	Альбедо спрайта до добавления света
Ambient fill	40, 44, 60	#282C3C	База, которую получает каждый пиксель даже без лампы
Light contribution	255, 214, 120	#FFD678	Аддитивный результат ламп, достающих до спрайта
SDF shadow	90, 70, 120	#5A4678	Часть вклада, которую снимает окклюдер или складка

Когда применять:

- Как базу до и после каждого изменения света, масок или политики коллизий. Исправление, которое выглядит правильно только в наложении, — не исправление.
- Когда спрайт неожиданно тёмный, чтобы понять, в чём причина: свет, тень или сам спрайт.
- Перед скриншотами и трейлерами: это единственный режим, совпадающий с релизной картинкой.

Как читать плохой результат:

- Тёмный спрайт обычно стоит за окклюдером, вне радиуса всех ламп или под низким ambient. Поднимите заполняющий свет, передвиньте лампу или проверьте список окклюдеров, прежде чем править цвет спрайта.
- Спрайт, который здесь выглядит правильно, а в Unlit пропадает, имеет проблему раскладки, а не освещения: см. следующий раздел.
- Выцветший спрайт означает, что его вклад света насыщен. Уменьшите интенсивность, сузьте конус или замените несколько декоративных свечений одной глобальной лампой.

2.2 Unlit / Wireframe

Авторское альбедо без расчёта освещения плюс границы каждого спрайта, скелетного меша и тканевого меша, нарисованные контуром. Режим нужен, чтобы оценивать текстуру, UV-прямоугольник, пивот и оттенок без заполняющего света и без тени, которая их скрывает.

Легенда и точные оттенки:

Строка легенды	RGB	Hex	Смысл на картинке
Unlit sprite	120, 220, 255	`#78DCFF`	Спрайт с авторским цветом и без освещения
Sprite bounds	80, 160, 255	`#50A0FF`	Квад обычного спрайта
Mesh or cloth bounds	60, 220, 180	`#3CDCB4`	Границы скелетного или тканевого меша

Когда применять:

- Перед настройкой света: проверять UV-прямоугольники, пивоты, размеры спрайтов и значения оттенка здесь дешевле, чем бороться с тенью, которая скрывает ошибку раскладки.
- После импорта нового атласа, чтобы увидеть, что каждый кадр берёт нужную область.
- Когда спрайт пропал в Standard, чтобы отделить проблему раскладки от проблемы освещения.

Как читать плохой результат:

- Пропал и здесь, и в Standard: маска рендера исключает проход спрайтов, альфа цвета равна нулю или у сущности нет компонента DrawOrder.
- Пустой прямоугольник там, где должен быть арт: альфа спрайта равна нулю или UV-прямоугольник попадает в прозрачную область атласа.
- Границы смещены относительно арта: неверен пивот или размер спрайта, а не текстура.
- Контур меша без арта: неверна палитра скина или количество вершин.

2.3 Collision Overlap

Сцена тонируется в серый, и каждый коллайдер рисуется цветом своего режима коллизий. Наложение читает ровно тот `CollisionMode`, который применяет физический мост, поэтому расхождение между нарисованным цветом и поведением — это настоящая ошибка конфигурации, а не визуальный артефакт.

Легенда и точные оттенки:

Строка легенды	RGB	Hex	Смысл на картинке
Simple collider (Box2D)	80, 220, 120	`#50DC78`	Полигон Box2D, статический или динамический
Complex SDF collider	250, 210, 70	`#FAD246`	Полигон Box2D, который дополнительно проверяет поле SDF
Trigger or query only	230, 90, 220	`#E65ADC`	Сенсор: сообщает о перекрытиях, никогда не толкает
No collision	140, 140, 150	`#8C8C96`	Нет ни политики, ни коллайдера, либо ступень NoCollision

Цвет берётся из `CollisionPolicyComponent` сущности. Сущность с `Collider2D`, но без политики рисуется как простой динамический коллайдер; сущность без того и другого рисуется как NoCollision.

Когда применять:

- При аудите уровня: каждая зелёная форма должна быть статической геометрией или движущимся объектом, каждая жёлтая — оправдывать проверку поля, каждая пурпурная — быть триггером, а каждая серая — быть назначенной вручную.
- При отладке «игрок проходит сквозь стену» и «игрока толкает триггер»: наложение показывает, какую ступень назначил автор.
- При проверке префаба или заспавненной волны, где унаследованные политики легко забыть.

Рекомендации по оптимизации:

- Держите геометрию уровня на простых статических формах. Сложный статический коллайдер платит за выборку поля расстояний при каждой проверке контакта и редко меняет поведение.
- Сложные динамические формы оставляйте персонажам и объектам, которым нужна реакция на склон или на тонкую стену.
- Убирайте NoCollision у спрайтов, которые всё ещё получают запросы перекрытия: они платят за запрос и никогда не могут ответить.
- Сворачивайте декоративные скопления в одну простую статическую форму вместо десятков мелких коллайдеров.

Полная лестница, включая то, что каждая ступень делает с телом и с фильтром формы, — в разделе 4.

2.4 Shader Complexity / Overdraw

Оценка работы на пиксель, которую получает спрайт, нарисованная одной непрерывной тепловой шкалой от зелёного через жёлтый к красному. Оценка умножает число квадов спрайтов, перекрывающих спрайт, на число достающих до него ламп и нормируется бюджетом сложности.

Строки легенды и их точные оттенки:

Строка легенды	RGB	Hex	Смысл на картинке
Low cost	80, 220, 120	`#50DC78`	Один слой заливки почти без света
Moderate cost	230, 230, 90	`#E6E65A`	Два-три перекрывающихся слоя
High cost	250, 150, 60	`#FA963C`	Четыре и более слоя или слои под несколькими лампами
Overdraw hotspot	230, 60, 60	`#E63C3C`	Насыщенный пиксель: бюджет достигнут или превышен

Оттенок вычисляется, а не выбирается из четырёх корзин, поэтому промежуточные цвета нормальны. Первая половина шкалы идёт от зелёного к жёлтому, вторая — от жёлтого к красному, а синий канал закреплён около 38, чтобы нагрев заменял цвет спрайта, а не просвечивал сквозь него. Значение считается так:

```
cost       = overlapping sprite count * (reaching lights + 1)
normalised = cost / viewport_complexity_budget
```

Нормированное значение 0 даёт чистый зелёный, 0.5 — чистый жёлтый, 1.0 и больше — чистый красный. Бюджет — это `viewport_complexity_budget`, по умолчанию `8`, с ограничением от 1 до 64: поднимите его, чтобы сцена выглядела холоднее, опустите, чтобы показать сцену тяжелее, чем она должна быть. Бюджет задаёт диагностическую опору и не меняет рендер.

Когда применять:

- При работе над производительностью fill-bound сцен: портреты поверх рисованных фонов, занавесы частиц, стопки свечений, десятки перекрывающихся декалей.
- Перед добавлением ещё одного полупрозрачного слоя в уже насыщенную сцену.
- Когда узкое место — GPU, а число draw call выглядит нормальным.

Как читать плохой результат:

- Красная полоса на весь экран обычно означает фоновый квад под всеми последующими слоями. Разбейте фон на области или рисуйте его один раз.
- Красные острова под скоплением ламп означают насыщение освещения, а не overdraw. Наложение складывает обе причины, поэтому перед сокращением арта посмотрите Light Complexity.
- Один белый спрайт, красный целиком: он перекрывает много квадов или несёт много ламп; два режима сложности вместе покажут, что именно.

Рекомендации по оптимизации:

- Убирайте перекрывающиеся полупрозрачные квады: каждый слой умножает стоимость заливки.
- Объединяйте мелкие спрайты в атлас, чтобы одна и та же область кадра заполнялась один раз.
- Отсекайте заэкранные и чисто фоновые объекты через `RenderMaskComponent`.
- Заменяйте несколько декоративных свечений одной глобальной или одной всенаправленной лампой.
- Снижайте бюджет только после того, как сцена понята: бюджет, который всегда показывает красное, ничему не учит.

2.5 Light Complexity

Число ламп, достающих до каждого спрайта после проверок радиуса и конуса.

Легенда и точные оттенки:

Строка легенды	RGB	Hex	Смысл на картинке
Ambient only	90, 140, 255	`#5A8CFF`	До спрайта не достаёт

			ни одна лампа
One light	80, 220, 120	`#50DC78`	До спрайта достаёт ровно одна лампа
Two or three lights	230, 230, 90	`#E6E65A`	До спрайта достают две или три лампы
Four or more lights	230, 60, 60	`#E63C3C`	Четыре и более ламп: тонмаппинг начинает сжимать светлые участки

Спрайты без света рисуются синим; остальные используют ту же тепловую шкалу, что и режим overdraw, с нормировкой числа ламп `(lights - 1) / 3`. Поэтому одна лампа — зелёная, две и три — жёлтые, четыре и больше — красные.

Счёт берётся из данных авторинга, а не из итогового цвета: он обходит включённые источники, пропускает лампу, чей владелец исключил проход источников света, применяет проверку радиуса и полную проверку конуса для spot и arc. Устаревшие радиальные лампы и глобальные лампы тоже участвуют, поэтому сцена, смешивающая два поколения компонентов, всё равно даёт честный счёт.

Когда применять:

- Когда спрайт ярче соседей и есть подозрение на лишний свет.
- Перед добавлением лампы в коридор, где их уже несколько.
- Когда нужно доказать, что статическая лампа классифицирована правильно и больше не участвует в покадровом расчёте.

Как читать плохой результат:

- Статическая лампа, которая ведёт себя как динамическая, обычно имеет снятый флаг «спящей статики», либо кэш статического вклада был сброшен движущейся целью или движущимся окклюдером. Счётчики из раздела 7 покажут, что именно.
- Спрайт красный, хотя экран выглядит тёмным: лампы до него достают, но их вклад перекрыт тенями. Проверьте окклюдеры.
- Спрайт синий внутри освещённой комнаты: его владелец исключил проход освещения, либо владелец лампы исключил проход источников света.

Замечание о счёте: наложение считает все включённые лампы, достающие до спрайта, включая лампы, помеченные статическими, и карточка помощи во вьюпорте говорит то же самое. Кэш статического вклада влияет на покадровый расчёт, а не на эту диагностику: сравнивайте по наложению спрайты между собой и воспринимайте его как меру нагрузки авторинга, а не как обещание о числе draw call.

Рекомендации по оптимизации:

- Помечайте лампы статическими, чтобы их вклад кэшировался до изменения сцены.
- Уменьшайте радиус: лампа, освещающая три комнаты, стоит как три комнаты.

- Сужайте конус у spot и arc. Всенаправленная лампа на полный оборот и узкая дуга стоят одинакового расчёта, но дуга достаёт до меньшего числа целей и легче балансируется.
- Предпочитайте «спящие статические» объекты и лампы, которые просыпаются только от контакта или по программному триггеру.

2.6 SDF Visualizer

Знаковое расстояние до ближайшей твёрдой поверхности, выбранное в каждой точке спрайта, поверх которого рисуются запечённые окклюдеры.

Легенда и точные оттенки:

Строка легенды	RGB	Hex	Смысл на картинке
Solid (inside the field)	230, 60, 60	`#E63C3C`	Точка выборки внутри твёрдой геометрии
Surface band	250, 210, 70	`#FAD246`	Ближе 16 мировых единиц к поверхности
Open space	80, 210, 220	`#50D2DC`	Далеко от любой поверхности или нет валидного поля
Baked occluder	160, 120, 255	`#A078FF`	Контур запечённого окклюдера, задающий тени и SDF-коллизии

Полоса поверхности фиксирована на 16 мировых единиц. Спрайт, нарисованный голубым в коридоре, означает, что валидных данных поля там нет или выборка далека от всех поверхностей; спрайт, нарисованный красным внутри комнаты, означает, что поле смещено или слишком грубое.

Когда применять:

- Перед тем как доверять теням: поле, расходящееся с артом, даёт тени, похожие на баги.
- При включении сложных коллизий на склоне, пандусе или тонкой стене.
- После правки геометрии уровня, чтобы убедиться, что поле пересобрано.
- При подборе размера ячейки поля: грубые ячейки делают жёлтую полосу ступенчатой, а красную внутренность — толстой.

Как читать плохой результат:

- Красное там, где арт показывает открытый пол: поле смещено или художник правил рельеф после запекания.
- Голубое там, где арт показывает стену: поле не пересобрано после правки или стена не помечена как окклюдер.
- Ступенчатая полоса поверхности: размер ячейки слишком груб для этой геометрии.
- Полное отсутствие цветовых различий: презентеру не передали валидное поле.

Рекомендации по оптимизации:

- Увеличивайте размер ячейки на больших открытых картах: поле — запечённый ассет, и его разрешение — компромисс между памятью и точностью.
- Пересобирайте поле только после правок, а не каждый кадр.
- Держите статический рельеф в запечённом поле, а движущимся объектам давайте динамические окклюдеры вместо принудительной пересборки.

3. Как выбрать режим и в каком порядке проверять

Продуктивная проверка идёт от данных к картинке, а не наоборот:

1. Collision Overlap: каждый коллайдер должен иметь задуманный уровень участия, иначе всё остальное неважно.
2. SDF Visualizer: поле должно быть согласовано с рисунком, иначе нельзя доверять сложным коллизиям и теням.
3. Unlit / Wireframe: арт, UV и пивоты должны быть верны, пока свет не оценён.
4. Light Complexity и Shader Complexity: найти спрайты и пиксели, которые стоят дороже, чем должны.
5. Standard: подтвердить, что исправление изменило релизную картинку.

Короткая таблица для частых вопросов:

Симптом	**Первый режим**	**Затем**
Спрайт слишком тёмный или светлый	Standard	Unlit для альбедо, Light Complexity для счёта ламп
Игрок проваливается сквозь стену	Collision Overlap	SDF Visualizer, если стена — сложный коллайдер
Тени выглядят неправильно	SDF Visualizer	Standard, чтобы оценить результат
Кадр упирается в GPU	Shader Complexity	Light Complexity, чтобы отделить свет от overdraw
Триггер толкает игрока	Collision Overlap	Сверить пурпурные формы со списком триггеров
Лампа не реагирует	Light Complexity	Проверить статические флаги и счётчики кэша

4. Лестница режимов коллизий

Участие в коллизиях задаётся `CollisionPolicyComponent`, а не формой. Политика отделена от `Collider2D`, чтобы дизайнер мог менять уровень участия, не переписывая форму или матрицу слоёв, и чтобы снимок состояния сохранял и то, и другое.

Поле	**Тип**	**По умолчанию**	**Смысл**
`Mode`	uint32	4 (SimpleDynamic)	Ступень лестницы, хранится числом, чтобы TOML, Mirror и инспектор обходились без кодека перечисления
`Category`	uint32	1	Биты категории формы в Box2D
`Mask`	uint32	0xFFFFFFFF	Биты маски в Box2D;

			для NoCollision записывается ноль
`Flags`	uint32	0	Зарезервированные биты авторинга

Семь ступеней по возрастанию участия:

Ступень	**Значение**	**Тело**	**Фильтр формы**	**Сенсор**	**Проверка поля**	**События контакта**	**Типичное применение**
NoCollision	0	Авторский тип сохраняется	Маска принудительно ноль	Нет	Нет	Нет	Чисто декоративный арт, пробники, которые должны проваливаться
QueryOnly	1	Авторский тип сохраняется	Категория и маска из политики	Да	Нет	Да	Запросы перекрытия и прямой видимости, подбираемые предметы
TriggerOnly	2	Авторский тип сохраняется	Категория и маска из политики	Да	Нет	Да	Игровые триггерные объёмы
SimpleStatic	3	Статическое	Категория и маска из политики	Нет	Нет	Да	Стены, пол, платформы в одну сторону
SimpleDynamic	4	Динамическое	Категория и маска из политики	Нет	Нет	Да	Ящики, враги, игрок
ComplexStatic	5	Статическое	Категория и маска из политики	Нет	Да	Да	Склоны, тонкие стены, точный рельеф
ComplexDynamic	6	Динамическое	Категория и маска из политики	Нет	Да	Да	Персонажи, которые должны следовать склонам

Чем ступени различаются:

- NoCollision не попадает в broad phase: маска фильтра формы равна нулю, поэтому контакт не возникает, а авторский тип тела сохраняется — это позволяет пробнику проваливаться сквозь геометрию. События не порождаются.

- QueryOnly и TriggerOnly — сенсоры: они сообщают о перекрытиях, не дают импульса и порождают события контакта. Мост обрабатывает их одинаково; лестница разделяет их, чтобы триггерный объём можно было найти, проверить и покрасить отдельно от пробника запросов. Оба рисуются пурпурным.
- SimpleStatic и SimpleDynamic разрешают контакты только полигоном Box2D. Это самые дешёвые ступени и правильный выбор по умолчанию для геометрии уровня и обычных объектов.
- ComplexStatic и ComplexDynamic дополнительно проверяют запечённое поле SDF — именно это заставляет склоны, пандусы и тонкие стены вести себя правильно. Они дороже на контакт и зависят от пересобранного поля.

Политику применяет `ApplyCollisionPolicyToCollider` до создания тел, поэтому смена режима не оставляет устаревший фильтр формы, а тело под управлением политики всегда включено: NoCollision убирает форму из broad phase, а не отключает тело.

Как выбрать ступень:

6. Начинайте всё как SimpleStatic или SimpleDynamic. Добавляйте сложные ступени только для поверхностей, где полигон заметно неверен.
7. QueryOnly — для пробников, которые не должны влиять на движение, TriggerOnly — для объёмов, на которые реагирует геймплей.
8. NoCollision оставляйте только декоративному арту и пересматривайте каждую серую форму, стоящую там, где может пройти игрок.
9. Перепроверяйте наложение Collision Overlap после добавления новой игровой системы: новые префабы наследуют политику своего шаблона.

5. Маски рендера

`RenderMaskComponent` позволяет сущности отказаться от проходов рендера, не отключая и не скрывая её. Это пара масок, а не одна, чтобы авторский пресет мог исключить проход, а более позднее переопределение — вернуть его.

Проход	Бит	Значение	Что закрывает
`RenderPassSprites`	0	1	Рисование спрайта и скелетного меша этой сущности, а значит и её диагностический оттенок
`RenderPassLighting`	1	2	Получает ли сущность результат света и участвует ли в кэше статического света
`RenderPassShadows`	2	4	Вычисляется ли выборка тени этой сущности
`RenderPassLightSources`	3	8	Участвует ли источник света этой сущности в расчёте и в счёте ламп
`RenderPassColliders`	4	16	Проход коллайдеров в

			извлечении наложений
`RenderPassOverlays`	5	32	Диагностические примитивы наложения этой сущности
`RenderPassSdf`	6	64	Вклады поля расстояний этой сущности
`RenderPassAll`	-	0xFFFFFFFF	Все проходы

Правило простое и полное: проход разрешён, когда его бит установлен в `Include` и снят в `Exclude`. `Exclude` побеждает, когда установлены оба, что делает позднее переопределение безопасным. `ExcludePass` добавляет бит в `Exclude`; `IncludePass` добавляет бит в `Include` и снимает его из `Exclude`.

Поле	Тип	По умолчанию	Смысл
`Include`	uint32	`RenderPassAll`	Проходы, в которых сущность может появляться
`Exclude`	uint32	`RenderPassNone`	Проходы, в которых сущность не появляется никогда

Практические рецепты:

Цель	Маска
Арт не виден во вьюпорте, но симуляция идёт	Исключить Sprites
Арт виден, но не освещается — для неосвещённого UI-слоя в мире	Исключить Lighting
Лампа нарисована, но пока не даёт вклада	Исключить LightSources у сущности лампы
Спрайт не должен получать и отбрасывать тени	Исключить Shadows
Вспомогательная форма только для диагностики	Исключить всё, кроме Overlays

Поскольку маска — это пара 32-битных целых, она сохраняется в TOML, в снимки и в инспектор без отдельного кодека, а префаб может нести значение по умолчанию, отличное от своих экземпляров.

6. Свет: один компонент, три формы

`LightSourceComponent` покрывает любую форму 2D-света. Тип определяет, как трактуется внешняя апертура; остальные поля общие.

Тип	Значение	Позиция и радиус	Конус	Примечания
Global	0	Игнорируются	Игнорируется	Добавляет свой цвет и интенсивность к базе после ambient; без затухания и без тени на цель

Omnidirectional	1	Используются	Принудительно полный оборот, независимо от `ConeAngle`	Обычная лампа
SpotArc	2	Используются	Использует `ConeAngle` и `ConeSoftness`	Узкая апертура — прожектор, широкая и мягкая — дуговая лампа

Поля рассчитываемого света:

Поле	**Тип**	**По умолчанию**	**Смысл**
`Color`	Color32	Белый	Цвет света; красный, зелёный и синий распаковываются по каналам
`Radius`	float	256	Мировые единицы, где вклад обнуляется; глобальная лампа его игнорирует, а решатель считает значения меньше 1 за 1
`Intensity`	float	1	Множитель после двух членов затухания; решатель ограничивает его снизу нулём
`Offset`	Vector2D	0, 0	Смещение, добавляемое к позиции трансформа владельца
`ConeAngle`	float	6.283185 (полный оборот)	Полная апертура в радианах; значения выше полного оборота ограничиваются им, значения ноль и меньше не дают света
`ConeSoftness`	float	0.35	Ширина полосы углового затухания в радианах, ограничена половиной апертуры
`Falloff`	float	2	Показатель затухания по расстоянию: 2 — близко к обратному квадрату, 1 — линейно
`Type`	uint32	1 (Omni)	Тип света из таблицы выше
`Flags`	uint32	Enabled, Static, Sleeping	Enabled, Static, Sleeping, CastNoShadow, WakeOnContact

Направление spot или arc — это `Transform2D.Rotation` владельца, поэтому поворот владельца наводит конус.

Поведение конуса, точно:

10. Половина апертуры — это `ConeAngle / 2`. Мягкость ограничивается этой половиной; апертура ноль и меньше возвращает ноль везде.
11. Апертура, равная полному обороту или больше, возвращает единицу везде — именно поэтому одна и та же функция обслуживает всенаправленные и конусные лампы.
12. По абсолютному углу между направлением лампы и направлением на получателя: угол на половине апертуры и дальше даёт ноль, угол внутри несмягчённого ядра даёт единицу, а полоса между ними использует smoothstep, поэтому на краю конуса нет видимого кольца.

Поведение по расстоянию: вклад равен `(1 - distance / Radius)` в степени `Falloff` и равен нулю за радиусом. Два предела разные: конус — это жёсткий срез с мягкой полосой, радиус — непрерывное затухание.

Что решатель делает с этими значениями:

Шаг	**Поведение**
Ambient	Цвет ambient, умноженный на интенсивность ambient, — это база
Глобальные лампы	Добавляются к базе после ambient
Omni, spot и arc	Расстояние, умноженное на конус и интенсивность, затем множитель SDF-тени, если тени включены и поле активно
Окклюзия	Отношение расстояния до твёрдого тела вокруг цели масштабирует статический вклад
Тонмаппинг	`x / (1 + x)` по каналам, поэтому светлые участки сжимаются, а не срезаются в белый
Излучение	Компонент цели может поднять результат к белому после тонмаппинга

Настройки решателя живут в `LightSourceSettings`:

Настройка	**По умолчанию**	**Смысл**
`Ambient`	40, 44, 60	Цвет заполняющего света
`AmbientIntensity`	0.25	Сила заполняющего света
`MaxLights`	128	Ёмкость массива выборок света на кадр
`Tonemap`	true	Применять `x / (1 + x)`
`SdfShadows`	true	Вычислять SDF-тени
`SdfMaxSteps`	16	Бюджет шагов марша на пару «лампа — цель», ограничен от 1 до 64
`SdfSoftness`	24	Ширина полутени теневой проверки
`SdfAmbientOcclusion`	0.35	Сила затемнения в складках
`SdfOcclusionRadius`	32	Радиус проверки складок
`CacheStaticLights`	true	Хранить статический вклад между кадрами

Для каждого поколения компонентов сцена использует один решатель: `LightSourceSystem` для `LightSourceComponent`, устаревший `LightingSystem` для

`Light2DComponent`. Оба пишут один и тот же компонент результата, поэтому извлечение кадра собирает результат, не зная, какой решатель его произвёл.

7. Статические и динамические лампы, жизненный цикл «спящей статики»

Классификация света — это три флага на компоненте:

Флаг	Бит	Смысл
`LightSourceEnabled`	1	Лампа участвует
`LightSourceStatic`	2	Вклад кэшируется до изменения сцены, целей, окклюдеров или поля
`LightSourceSleeping`	4	Статическая лампа может проснуться; без статического бита не действует
`LightSourceCastNoShadow`	8	Лампа никогда не вычисляет теневую проверку
`LightSourceWakeOnContact`	16	Контакт может разбудить спящую статическую лампу

Лампа с битом Static и без Sleeping — постоянно статическая. Лампа с обоими битами — спящая статическая: она даёт вклад через кэш, но просыпается в динамический расчёт при первом разрешённом контакте или программном триггере.

Пробуждение одной лампы — `WakeLightSource`:

- Возвращает false, когда лампа не спит. Динамическую лампу нельзя разбудить дважды.
- Возвращает false, когда пробуждение пришло от контакта, а разрешения wake-on-contact нет.
- Иначе снимает биты Static и Sleeping, и следующий расчёт трактует лампу как динамическую.

Кэш статического вклада — причина дешевизны статических ламп. Каждый кадр решатель строит подпись из данных, которые могут изменить статический результат:

- позиция, направление, радиус, интенсивность, цвет, апертура, мягкость и затухание каждой статической лампы;
- центр и радиус каждого динамического окклюдера;
- настройки мягкости тени и ambient occlusion;
- идентичность поля SDF;
- идентификатор и позиция каждой цели освещения, потому что движущийся получатель требует пересборки, даже если все лампы остались на месте.

Если подпись совпадает с сохранённой и кэширование включено, решатель переиспользует сохранённый вклад и увеличивает `StaticCacheHitCount`. Иначе он пересчитывает все цели и увеличивает `StaticRebuildCount`. `InvalidateStaticCache`

принудительно вызывает пересборку на следующем расчёте; загрузка сцены, телепорт целой комнаты или отладочный переключатель — естественные места вызова.

Два счётчика — самый дешёвый способ убедиться, что кэш работает: коридор со статическими лампами должен показывать растущее число попаданий и число пересборок, которое останавливается после стабилизации сцены. Число пересборок, растущее каждый кадр, означает, что подпись меняется каждый кадр: движущаяся цель, движущийся окклюдер или пересоздаваемый указатель на поле.

«Спящие статические» сущности — общая форма той же идеи, и она шире, чем свет: штабель ящиков, коллизионный декор или лампа могут быть статическими, пока их что-то не коснётся.

Флаг	**Бит**	**Смысл**
`kSleepingStaticFlagActive`	1	Компонент активен
`kSleepingStaticFlagSleeping`	2	Сущность сейчас статична и спит
`kSleepingStaticFlagWakeOnContact`	4	Контакт может её разбудить
`kSleepingStaticFlagWakeOnTrigger`	8	Триггер может её разбудить
`kSleepingStaticFlagLight`	16	Владелец также источник света, поэтому пробуждение переклассифицирует лампу

Флаги по умолчанию — активный, спящий и пробуждение по контакту: декор, который статичен, пока его не коснутся, — именно то состояние авторинга, которое нужно большинству объектов. `WakeCount` и `LastWakeReason` хранят историю для инспектора и панели диагностики. Причины: None, Contact, Trigger, Program и SceneEdit.

Жизненный цикл:

Состояние	**Событие**	**Результат**
Спит, разрешено пробуждение по контакту	Контакт типа BeginTouch или Hit	Флаг сна снят, причина Contact, счётчик увеличен, тело переключено в динамическое и разбужено, лампа переключена в динамическую
Спит, разрешено пробуждение по триггеру	Программное или триггерное пробуждение	Флаг сна снят, причина Trigger или Program
Проснулась	Ничего	Остаётся проснувшейся: пробуждение одностороннее до правки сцены
Проснулась	Правка сцены	`ResetSleepingStatic` возвращает флаги активного и спящего состояния и очищает причину
Динамическая лампа	`WakeLightSource`	Возвращает false: будить нечего

Пробуждение одностороннее, потому что от этого зависит детерминированность: контакт на тике N будит сущность при каждом повторе тика N. Симуляция, которая сама усыпила бы сущность обратно, зависела бы от тайминга кадров и разошлась бы.

Практический рецепт для освещённой комнаты:

13. Задайте каждую лампу статической, с флагом сна и разрешённым пробуждением по контакту. После первого расчёта комната не стоит ничего покадрово.
14. Будите только те лампы, которые игрок может разбить или включить, программным триггером, а не контактом, чтобы пробуждение не зависело от геометрии столкновений.
15. Дайте декору `SleepingStaticComponent` со флагами по умолчанию и позвольте контактам его будить.
16. Следите за `StaticRebuildCount`: если он растёт, когда ничего не движется, значит что-то в подписи меняется каждый кадр.

8. Остановка времени и замедление

Есть два независимых механизма, и они перемножаются.

Глобальный — `PhysicsSystem::SetTimeScale`. Он масштабирует шаг мира: в Box2D передаётся фиксированная дельта, умноженная на масштаб. Масштаб 1 — реальное время, 0.5 — половинная скорость, 0.25 — bullet time. Масштаб ноль и меньше останавливает мир: шаг полностью пропускается, поэтому ничего не интегрируется и не возникает событий контакта, но система всё равно применяет политики представления, синхронизирует тела и создаёт недостающие.

Локальный — `TimeScaleComponent`:

Поле	Тип	По умолчанию	Смысл
`Scale`	float	1	Множитель часов одной сущности
`Reserved`	float	0	Зарезервировано

Масштаб сущности применяется на каждом физическом шаге к каждой сущности, у которой есть и компонент, и `RigidBody2D`:

- Масштаб ноль замораживает тело: линейная и угловая скорости обнуляются, тело Box2D отключается, поля скоростей очищаются, тело помечается замороженным. Замороженная сущность не интегрируется и не получает импульсов.
- Положительный масштаб размораживает тело: тело Box2D включается снова, если сущность тоже включена, а отметка заморозки снимается.
- Авторские импульсы, переданные через `ApplyImpulse`, умножаются на масштаб, а импульс по замороженной сущности игнорируется.
- Отсутствие компонента означает масштаб 1, поэтому поведение не меняется, пока вы не зададите часы.

Что даёт комбинация:

Эффект	Глобальный масштаб	Масштаб сущности
Пауза удара для одного персонажа	1	0 у этой сущности
Замедление всей комнаты	0.25	1
Bullet time с исключением для игрока	0.25	4 у игрока, чтобы компенсировать глобальное замедление
Полная остановка для катсцены	0	1
Замороженный декор, который никогда не проснётся	1	0

Два предостережения. Во-первых, масштаб времени принадлежит физике: анимация, аудио, UI и таймеры скриптов ведут собственные часы, поэтому остановленный мир может продолжать анимировать курсор или играть музыку. Во-вторых, замороженное тело — не то же самое, что спящее: сон отдан Box2D и сохраняет тело для последующего пробуждения, а замороженное тело отключено и остаётся отключённым, пока его масштаб не станет положительным.

9. Палитра SDF-примитивов

В палитре двадцать точных семейств знаковых расстояний. Форма задаётся в `SdfShapeComponent`, который оборачивает один `SdfShape`. Семейство определяет, какие поля читает формула расстояния; остальные поля игнорируются.

Семейство	Значение	Какие поля читает	Значение в палитре
Circle	0	`Radius`	радиус 16
Box	1	`HalfExtents`	16 на 16
RoundedBox	2	`HalfExtents`, `Radius` как радиус скругления	16 на 16, скругление 4
Capsule	3	`HalfExtents` как половина хорды, `Radius` как толщина	хорда 16, толщина 8
Segment	4	`HalfExtents` как половина хорды	хорда 16
Ellipse	5	`HalfExtents` как два радиуса	20 на 12
Rhombus	6	`HalfExtents` как половины диагоналей	18 на 12
EquilateralTriangle	7	`HalfExtents.Y` как радиус описанной окружности	16
IsoscelesTriangle	8	`HalfExtents.X` половина основания, `HalfExtents.Y` половина высоты	16 на 16
Trapezoid	9	`HalfExtents.X` половина нижней ширины, `HalfExtents.Y` половина высоты, `Width` половина верхней ширины	16 на 12, верх 8
Parallelogram	10	`HalfExtents`, `Width` как горизонтальный	16 на 12, сдвиг 6

		сдвиг	
Hexagon	11	`HalfExtents.Y` как радиус описанной окружности	16
Octagon	12	`HalfExtents.Y` как радиус описанной окружности	16
Star	13	`HalfExtents.X` внутренний радиус, `HalfExtents.Y` внешний радиус, `Sides` число лучей	внутренний 8, внешний 18, 5 лучей
Pie	14	`Radius`, `Width` как половина апертуры	радиус 16, половина апертуры 0.785398
Ring	15	`Radius` средний радиус, `HalfExtents.X` половина толщины	средний 16, толщина 4
Arc	16	`Radius` средний радиус, `HalfExtents.X` половина толщины, `Width` половина апертуры	средний 16, толщина 3, половина апертуры 0.785398
Cross	17	`HalfExtents` как половины двух плеч, `Radius` как радиус скругления	16 на 6
Moon	18	`Radius` внешний радиус, `HalfExtents.X` смещение отсекающей окружности, `HalfExtents.Y` радиус отсечения	внешний 16, смещение 8, отсечение 14
Heart	19	`Radius` как общий размер	16

Правила по полям, которые полезно знать до авторинга:

- `Angle` поворачивает запрос вокруг начала координат в радианах, поэтому форма поворачивается, не смещая центр.
- `Width` — вторичный скаляр, смысл зависит от семейства: апертура для pie и arc, половина верхней ширины для трапеции, горизонтальный сдвиг для параллелограмма. Для pie и arc значение ограничивается от 0 до pi/2.
- `Sides` читается только звездой и ограничивается от 3 до 32.
- `Flags` зарезервирован под подсказки авторинга и рендера и никогда не меняет расстояние.
- Компонент занимает 32 байта, тривиально копируем и имеет стандартную раскладку, а палитра индексируется по семейству, поэтому `MakeSdfShape` возвращает копию точной строки значений по умолчанию.

Три запроса:

Функция	Результат
`SdfDistance`	Точное знаковое расстояние до границы: отрицательное внутри, положительное снаружи
`SdfGradient`	Единичный внешний градиент через центральные разности с шагом 1e-3; ноль там, где разность исчезает
`SdfShapeCoversPoint`	Истинно, когда точка строго внутри

Каждое семейство точное, включая эллипс, который вычисляет замкнутую квартальную форму вместо итераций. Недопустимое семейство в описании возвращает строку Circle, поэтому испорченный тип никогда не даёт неконечное расстояние.

Использование палитры вместе с остальным руководством:

- Стройте форму из `SdfPrimitivePalette` или `MakeSdfShape`, затем правьте поля под семейство. Писать функцию расстояния для новой формы руками — неверный шаг: это ломает контракт палитры, инспектор и запекание поля.
- Сложные ступени коллизий проверяют то же поле, которое рисует SDF Visualizer. Если сложный коллайдер ведёт себя странно, сначала посмотрите на SDF Visualizer.
- Палитра предназначена для коллизионных форм и диагностического рисования. Она не заменяет арт спрайтов.

10. План оптимизации

Симптом	Куда смотреть	Обычное исправление
В Shader Complexity всё красное	Перекрывающиеся полупрозрачные квады	Сократить, объединить или замаскировать слои; один фоновый квад вместо нескольких
Красно только под скоплением ламп	Light Complexity	Меньше ламп, но шире; декоративные лампы пометить статическими
Статические лампы всё ещё заметно стоят	Счётчики кэша	Убрать движущуюся цель или окклюдер, вызывающий пересборку
Всплеск времени кадра при падении объекта	Ступени коллизий	SimpleStatic для пола, сложные только там, где полигон неверен
Тени расходятся с артом	SDF Visualizer	Пересобрать поле, пометить стену как окклюдер
Триггер толкает игрока	Collision Overlap	Ступень — SimpleDynamic, а не TriggerOnly
На каждый спрайт светит каждая лампа	Значения радиуса и конуса	Уменьшить Radius, сузить ConeAngle у spot и arc
Много серых коллайдеров на уровне	Ступень NoCollision	Вернуть SimpleStatic там, где есть стена

11. Настройки, файлы и связанные документы

Два ключа настроек, от которых зависит это руководство:

Ключ	Поле	По умолчанию	Диапазон	Смысл
`viewport_view_mode`	`viewportViewMode`	0	от 0 до 5, недопустимые значения приводятся к 0	Активный режим, хранится в файле проекта и дублируется в INI редактора
`viewport_complexity_budget`	`viewportComplexityBudget`	8	от 1 до 64	Опорное значение нормализации тепловой шкалы Shader Complexity

Связанные документы:

- [Справочник настроек редактора](EditorSettingsReference_RU.md): все ключи редактора, их значения по умолчанию, ограничения и дублирование в INI.
- [Lighting](../../design/lighting.md): путь CPU-освещения, SDF-тени и GPU-маска вычислений.
- [Time](../../design/time.md): часы, таймеры и кадровый тайминг вне физического шага.
- [World systems](../../design/world-systems.md): тайлмапы, скелетный IK и навигация, которые используют лестницу коллизий и маски рендера.
- [Рекомендации по настройкам проекта](ProjectSettingsRecommendations_RU.md): как числа редактора вписываются в профиль проекта.
- [Application Guide](FemboyEngineApplicationGuide_RU.md): процесс авторинга, который использует эту диагностику.